

Information Technology



Practical Assessment Task (PAT) Grade 12 – 2026

Assessment Rubric

LEARNER EXAM NO.:	
LEARNER NAME:	

Details	Marks	Learner Mark
Specifications Document	15	
Design Document	30	
Coding	40	
Technical & Testing Document	15	
TOTAL	100	

1. SPECIFICATIONS DOCUMENT (See SAGs Appendix G – 12.4.16)		MAX	ACTUAL	COMMENTS
1.1 Problem Summary A brief description of the program including the purpose of the program, summary of functions and description of target user group(s)		4		
[4] Purpose, summary of functions and target user groups, well-described.				
[2–3] Aspects mostly well-described, with at least one not completely specified/described.				
[1] Only one aspect well described or part of each not fully described.				
[0] Aspects not described or completely inadequate.				
1.2 Research and Motivation Research is done to discuss similar programs. Research is correctly referenced. A motivation is included for the program indicating how this program differs from existing programs and why candidate chose to code this program.		3		
[3] Similar programs are described and referenced. A motivation is supplied to include an explanation of how the proposed program will differ from existing programs.				
[1–2] Incomplete research, referencing or insufficient motivation. Deduct one mark for each incomplete aspect.				
[0] No research or motivation provided.				
1.3 Program Functions (See SAGs Taxonomy 2.6.2) A detailed function list of what the program will do, grouped according to user group(s).		5		
[4–5] Function list for scenario is detailed and complete, for all user group(s), with sufficient complexity for the proposed program.				
[2–3] The function list is a substantial list of appropriate functions but is insufficient in complexity; or the functions list is not complete; or not completely specified for user group(s).				
[1] The function list results in a simplistic program.				
[0] No functions listed.				
1.4 Data Required		3		
[3] All data for given scenario and the stipulated program functions, have been correctly described.				
[1–2] Some data, for given scenario and the stipulated program functions, are described with a few errors.				
[0] No data provided for given scenario and the stipulated program functions.				
TOTAL		15		

2. DESIGN DOCUMENT (See SAGs Appendix G – 12.4.16)	MAX	ACTUAL	COMMENTS
2.1 Program Flow Diagram The candidate must use any form of diagram to present a global overview of the program and how the program is used. It also needs to take into account user group(s) and include navigation, and pathways.	5		
[4–5] Diagram is clear and easy to understand. There are no logical gaps on how the program is used, based on user group(s).			
[2–3] Diagram is substantial but still has some logical gaps on how the program is used.			
[0–1] No diagram and/or large logical gaps in the use of the program.			
2.2 User Interface Description (See SAGs Appendix G – 10.2.5, 10.4.12, 11.4.12) GUI screens for the program functions specified in section 1.3, must be described, to include: • Theme, colours, font, font size used for screens and for the program. • The purpose of each screen. • List of GUI components on each screen with an indication of what these components are used for. • Appropriate choice of GUI components, used for data values, from the user.	6		
Theme, colours, fonts, and font sizes clearly described. [1]			
Purpose of each screen is fully explained. [2]			
Complete and accurate list of GUI components provided for every screen with clear indication of their use. [2]			
GUI components are appropriately chosen, intuitive, and suitable for data input and user interaction. [1]			
2.3 Secondary Storage Design (See SAGs Appendix G – 11.4.7, 12.4.7, 10.4.10, 12.4.10) The candidate must show how data will be permanently stored. • Storage of data can be local, remote or cloud based, with justification. • Storage of data could be done using either a database, text files, JSON, or a combination thereof. • Structures must show data, logically separated. • Fields must be correctly named and described, to include the type of data (where applicable). • Sample data for each logical separation must be provided.	5		
Permanent storage method (local, remote, or cloud), indicated and justified. [1]			
Appropriate structure (database, text files, JSON, or a combination) used. [1]			
All data logically separated (tables, files, or collections well defined), clearly illustrating structure. [1]			
Fields are correctly named and described with data types included (string, integer, date, etc.). [1]			
Sample data is provided for each logical separation. [1]			
2.4 Explanation of Secondary Storage Design (See SAGs Appendix G – 12.4.7, 12.4.10) The candidate must provide an explanation for their choice of secondary storage design. The explanation must demonstrate a justification of the secondary storage design and an understanding of the implications of the chosen design as opposed to other secondary storage designs.	3		
[3] Explanation shows an in-depth understanding of the implications of the secondary storage design and is completely justified.			
[2] Explanation is substantial, but it is not completely justified. There are some areas of confusion or lack of understanding of the implications of the storage design.			
[0–1] No explanation of secondary storage design is provided or no evidence of understanding of the storage design.			

2.5 Class Diagrams for Backend Classes (See SAGs Appendix G – 11.4.3, 12.4.3, 11.4.5, 12.4.5) The candidate must provide class diagrams with a class name, fields, and methods demonstrating the application of OOP principles, for all Backend classes.			
Fields and methods are logically separated, into classes, with each class diagram correctly structured. [2]	8		
Classes implement hiding information. Fields and methods have appropriate access modifiers. [1]			
Fields are meaningfully named with appropriate data types. [1]			
Method lists contain the relevant constructors, accessor, mutator, helper, auxiliary and toString methods. [1]			
Methods are meaningfully named, appropriately typed, with correct parameters (where applicable). [1]			
Class diagrams are thorough and clearly relate to the program functions in section 1.3. [2]			
2.6 Explanation of which Backend classes relate to Secondary Storage (See SAGs Appendix G – 10.4.3, 11.4.3, 12.4.3, 11.4.7, 12.4.7) Description of which Backend classes provided as class diagrams in section 2.5, will represent the data stored in secondary storage provided in section 2.4, and how these classes will be used to handle data to and from secondary storage.			
[3] A clear and detailed representation of which class relates to which data stored in secondary storage, and how this data will be represented, when the data is read or written to secondary storage, using the backend classes.	3		
[1–2] Some form of representation of which class relates to which data stored in secondary storage and how this data will be represented, when the data is read or written to secondary storage, using the Backend classes. However, some details are missing or incomplete.			
[0] No representation of which class relates to which data in secondary storage.			
TOTAL	30		

3. CODING The program must be assessed by examining the source code in conjunction with an interview with the candidate. The program must also be run to determine whether it achieves the functions listed in section 1.3.	MAX	ACTUAL	COMMENTS
3.1 Secondary Storage Implementation (See SAGs Appendix G – 10.4.10, 12.4.10, 11.4.7, 12.4.7) Evaluate secondary storage provided for the program against the design provided in section 2.3.	3		
[3] Implemented fully as described in section 2.3.			
[1–2] Implemented, but not fully as described in section 2.3.			
[0] Not implemented.			
3.2 Implementation of Backend Classes (See SAGs Appendix G – 11.4.5, 12.4.5) Backend classes are correctly created and coded, as per class diagrams provided in section 2.5.	4		
[3–4] Implemented fully as described in section 2.5.			
[1–2] Implemented, but not fully as described in section 2.5.			
[0] Not implemented.			
3.3 Comments (See SAGs Appendix G – 12.4.15) Only required for the TWO most significant Backend classes. Code has comments using an API and/or comments, which are placed inside source code to explain code, fields, methods/functions, parameters and return types.	3		
[3] TWO most significant Backend classes have comments. All methods/functions have comments describing what they do. Comments include the data that the methods/functions return (for typed methods/functions) and the data they receive (parameters). Steps for complex algorithms have comments.			
[1–2] At least ONE significant Backend class has comments. Not all methods/functions have comments. Not all parameters and return types have comments. Comments are brief and contain little relevant detail.			
[0] No comments			
3.4 User Interface Implementation (See Sags Appendix G – 10.2.5, 10.4.12, 11.4.12) Screens are created according to descriptions provided in section 2.2 and are connected together, as illustrated in section 2.1.	6		
Theme, colours, fonts, and font sizes described in section 2.2, have been consistently applied to screens. [1]			
Screens contain the required GUI components, as described in section 2.2. [2]			
Screens are connected together, as illustrated in section 2.1. [2]			
Position of screens and the layout/grouping of components on the screens, are consistent from screen to screen. [1]			
3.5 Separation of Working Code (See SAGs Appendix G – 11.4.5, 12.4.5)	6		
[5–6] All working code is done in Backend classes and not UI classes. The code in UI classes focuses on UI elements and use methods in Backend classes for functionality and to access data in secondary storage.			
[3–4] Most of the working code is done in Backend classes. Most of the code in UI classes, focuses on UI elements. UI classes do contain some working code that could have been done in Backend classes. UI classes access some secondary storage data directly.			
[1–2] Majority of working code is done in UI classes. UI classes make little use of Backend classes. UI classes access most of the data in secondary storage, directly.			
[0] All working code in UI classes. UI classes make no use of Backend classes. UI classes access all data in secondary storage, directly.			

3.6 Good General Programming Techniques (See SAGs Appendix G – 10.4.6, 11.4.6, 12.4.6, 10.4.8, 11.4.8, 12.4.8)			
[5]	Code is technically perfect. Indentation immaculate. Variable names are all descriptive and follow conventions. Programming structures are appropriate e.g. <i>switch</i> or case instead of multiple <i>if</i> statements, duplication of code is eliminated using appropriate structures such as arrays or methods.	5	
[1–4]	Errors in programming techniques (–1 per error type – multiple instances of the same error do not accumulate deductions). Errors include: No indentation, single level indentation, inconsistent or inaccurate indentation, variable names do not clearly indicate what the variable is used for, multiple variables used instead of arrays, multiple <i>if</i> statements instead of switches or case statements, repetition of code (instead of using a typed method/function/void method/procedure or arrays).		
[0]	Code does not incorporate any good general programming techniques.		
3.7 Defensive Programming (See SAGs Appendix G – 11.4.7, 11.4.8, 11.4.14) Data validation, exception handling and error messages.			
[4]	Data is controlled and validated using code or appropriate UI components. Candidate needs only to code once, for every data type instance, where the choice of UI component does not assist with validation. Potential major IO and mathematic errors protected/trapped with relevant exception handling. All error messages are descriptive and easy to understand.	4	
[3]	Most data are controlled and validated using code or appropriate UI components. There are few instances where candidate has not coded for every data type instance, where UI component does not assist with validation. Potential major IO and mathematic errors protected/trapped with relevant exception handling. Vague or insufficient error messages.		
[1–2]	Few data are controlled and validated using code or inappropriate UI components used. There are many instances where a candidate has not coded for every data type, for instance, where the UI components used do not assist with validation. Potential major IO and mathematic errors not protected/trapped with relevant exception handling. No error messages.		
[0]	No data validation and exception handling.		
3.8 Fulfilment of Specifications The program must be tested against the functions listed in section 1.3. This can only be assessed by running the compiled program. Cross reference with the functional testing done in section 4.3.			
[5–6]	The program satisfies a detailed and complete function list with code that works correctly, for all user group(s), with sufficient complexity for the scenario.	6	
[3–4]	The program satisfies a substantial list of appropriate functions but is insufficient in complexity or the functions are not completely coded or correct for the specified user group(s).		
[1–2]	Simplistic program, with basic functionality. Missing functions or significant number of functions do not work.		
[0]	The program does not execute or does not have any functionality.		
3.9 User Experience (See SAGs Appendix G – 10.2.5, 10.4.12, 11.4.12) This can only be assessed by running the compiled program.			
[3]	Program is easy to use, navigate and understand: an excellent user experience.	3	
[2]	Sufficient level of user interface present, to provide a good user experience for the program. Some instances of unnecessarily complex navigating and/or aspects design that are confusing to the user.		
[1]	The user is lost – does not know where to start or how to use the program, or user interface present but not at a sufficient level for the user to engage with.		
[0]	Program does not execute or there is no user interface to engage with.		
TOTAL		40	

4. TECHNICAL & TESTING DOCUMENT (See SAGs Appendix G – 12.4.16)	MAX	ACTUAL	COMMENTS
4.1 Externally Sourced Code Provide a list of ALL external code that were adapted, copied or referred to, when creating this program. All AI Generative code used, must include the tools and prompts used. No more than 20% of the program code may be from external sources or from Generative AI. External code can be confirmed with an interview, incorporating oral review of code and techniques.	3		
[3] List of all external code provided, with sources and the purpose for each piece of externally sourced code. Tools and prompts for AI generative code also discussed.			
[1–2] List of all external code provided, however sources and/or purpose for each piece of externally sourced code is missing or inadequate. Tools and prompts for AI generative code not adequately discussed.			
[0] List not provided.			
4.2 Critical Algorithms Provide pseudocode and explanation of algorithm(s) that are vital for the program's functioning. Programming code is NOT acceptable.	3		
[3] Pseudocode is correct and clear explanation(s) as to why algorithm(s) is critical.			
[1–2] Pseudocode contains errors and/or explanation(s) as to why algorithm(s) is critical, or is not clear.			
[0] No pseudocode and explanation(s).			
4.3 Functional Testing: At least TWO sets of functional testing evident together with the tester's name, the date the testing was performed and the result of each functional test. Each test should indicate whether the program satisfies the functions listed in section 1.3. It is acceptable if some of the functions are not working, as long as progression is seen through the testing process.	3		
[3] TWO sets of functional tests and all requirements tested with all details present.			
[1–2] Not all requirements were tested and/or not sufficiently described: missing details such as when, with whom and the result.			
[0] No testing, not indicated or original function list was insufficient (or not present).			
4.4 Test Plan and Results for TWO input variables (See SAGs Appendix G – 10.4.13, 11.4.13) The TWO input variables must be clearly identified from 3.7. Testing should be done using standard, extreme and abnormal data. Screenshots showing before and after of each test for each variable must be included.	3		
[3] Full test plan with correct data for TWO input variables which are clearly identified, together with screenshot results.			
[1–2] Some test plan and screenshot results present but at least one element is missing.			
[0] No test plan and no screenshot results present.			
4.5 Evaluation of the Program An objective evaluation on how the program satisfies the functions listed in section 1.3. Suggestions must be included on how to address failures, should functions not be met. Alternate solutions or improvements must also be included.	3		
[3] Evaluation is thorough and includes suggestions for all shortfalls and/or suggestions for improvements.			
[1–2] Evaluation includes suggestions for some shortfalls and/or suggestions for improvements, not clearly explained.			
[0] No evaluation.			
TOTAL	15		



INFORMATION TECHNOLOGY PRACTICAL ASSESSMENT TASK DECLARATION

Learner Exam Number	
----------------------------	--

DECLARATION BY THE LEARNER:

I, _____ (learner full name)
hereby confirm that:

- I have read and understood my school's policy relating to Academic Dishonesty.
- I have adhered to my school's policy regarding Academic Dishonesty.
- The PAT submitted has been completed in full by me.
- I have clearly indicated in any sections of work/code that were generated or assisted by AI or where code was borrowed, giving credit where it is due.
- I acknowledge that I may have to verbally explain my work presented in this PAT.
- I further declare that my PAT code comprises 20% or less code that is borrowed or from Generative AI, as required by the IEB, and this is clearly indicated in my documentation.

Signed: _____ Date: _____
Learner

DECLARATION BY TEACHER:

I, _____ (teacher full name & title)

at _____ (name of school)

declare that the work provided by this learner has been monitored and checked for plagiarism.

Signed: _____ Date: _____
Teacher